

SYSTEM ANALYSIS DOCUMENTATION (SAD)

AI IncidentFlow – Agentic AI Incident Orchestration Platform

Document Prepared by: We-cooked-nen (Group 112)

Group Members:

Chiew Yi Jia

Tan Yi Shen

Soo Desmond

Yan Calvin

Yau Han Hong

Table of Contents

Table of Contents.....	2
1. Introduction.....	3
1.1 Purpose.....	3
1.2 Background.....	3
1.3 Target Stakeholders.....	4
2. System Architecture & Design.....	4
2.1 High Level Architecture.....	4
2.1.1 Overview.....	5
2.1.2 LLM as Service Layer (Agentic Workflow & Context Caching).....	5
2.1.3 Dependency Diagram.....	6
2.1.4 Sequence Diagram (Multi-Turn Reasoning Loop).....	7
2.2 Technological Stack.....	7
2.2.1 Frontend.....	7
2.2.2 Backend.....	7
2.2.3 Database.....	7
2.2.4 AI Service.....	7
3. Functional Requirements & Scope.....	7
3.1 Minimum Viable Product.....	7
3.2 Functional Workflow.....	8
3.3 Non-Functional Requirements.....	8
3.4 Out of Scope / Future Enhancements.....	9
4. Monitoring, Assumptions & Dependencies.....	9
4.1 Monitoring.....	9
4.2 Assumptions.....	10
4.3 External Dependencies.....	10
5. Project Management & Team Contributions.....	10
5.1 Project Management.....	11
5.2 Recommendations.....	11
6. AI declaration.....	12

1. Introduction

1.1 Purpose

This System Analysis Documentation describes the technical scope, workflow design, and system architecture of AI IncidentFlow, an advanced AI-powered incident ticket orchestration platform designed to automate incident triage, ticket routing, and escalation workflows across multiple enterprise departments.

The purpose of this system is to transform unstructured incident submissions from support portals and simulated emails into structured tickets that are intelligently classified, prioritized, assigned, linked to similar historical incidents, and tracked throughout their lifecycle.

The key components covered in this document include:

1. Architecture: Client-server web application with an Agentic AI reasoning service integration and Context Caching.
2. Workflow Logic: Multi-turn AI reasoning loop involving tool calling for server status checks, ticket summarization, classification, assignment, and escalation.
3. State Management: Ticket lifecycle tracking and relational linking of similar incidents within the SQLite database.
4. System Scope: Definition of MVP functionality for incident submission, real-time ticket log monitoring, and AI analysis transparency.

This document serves as the primary technical reference for system design, implementation planning, and MVP development.

1.2 Background

Organizations frequently receive operational incident reports from staff through emails, internal portals, or manual communication channels. These incident reports are often unstructured, incomplete, and require manual triage to determine urgency, responsible teams, and resolution steps.

AI IncidentFlow addresses these issues by integrating Z.AI GLM as an active reasoning agent to:

- Interpret incident descriptions
- Autonomously trigger backend tools to verify real-time system health (e.g., checking if the payroll server is down)
- Contextually apply enterprise routing policies via a static Knowledge Base

- Summarize incidents and output guaranteed structured JSON data
- Classify ticket categories and assign responsible teams
- Link the new ticket to previously reported similar incidents (Similar Incident Discovery)
- Monitor workflow states and trigger SLA-based escalations

1.3 Target Stakeholders

Stakeholder	Role	Expectations
Employee / Client	Submit incidents through support portal or email	Quick and simple incident submission with immediate ticket generation.
Support Teams	Receive and resolve assigned tickets	Accurate categorization, real-time diagnostic context gathered by AI tools, linked related tickets, and clear incident details.
Managers	Monitor escalated incidents & AI reasoning	Visibility into unresolved tickets, full transparency into the AI's triage logic via the dashboard, and escalation logs.
Development Team	Build and maintain the platform	Modular architecture, manageable AI integration using strict JSON schemas and tool definitions.
Judges / Evaluators	Assess workflow orchestration capabilities	Clear demonstration of Agentic AI reasoning, multi-turn tool calling, context caching, and robust workflow automation.

2. System Architecture & Design

2.1 High Level Architecture

2.1.1 Overview

Type	Details
System Architecture	Client-Server Web Application with Agentic AI Service Layer

The system follows a client-server architecture consisting of two primary frontend pages:

1. Incident Submission Page: Allows users to submit incidents via support form or simulated email interface.
2. Ticket Log Dashboard: Displays an overview of tickets and provides an in-depth modal revealing the AI's full reasoning, priority generation, summary, and dynamically linked related tickets.

The backend provides REST API endpoints for:

- Receiving incident submissions
- Initiating the multi-turn agentic loop with the AI reasoning engine
- Executing internal tools requested by the AI (e.g., `check_server_status`)
- Storing tickets and their linked relationships in the database
- Fetching ticket details and AI analysis for the dashboard
- Escalating unresolved incidents automatically based on SLA thresholds

2.1.2 LLM as Service Layer (Agentic Workflow & Context Caching)

The AI reasoning layer acts as an autonomous agent rather than a passive text analyzer.

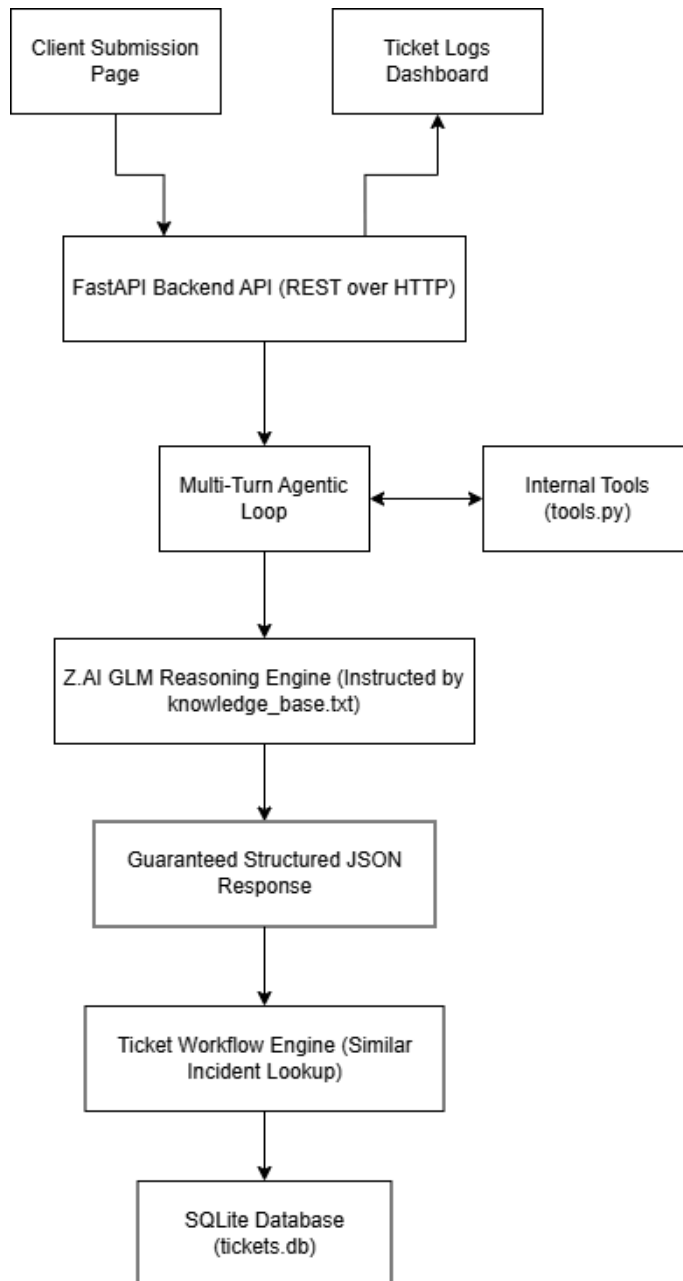
Context Caching: The backend injects a static enterprise Knowledge Base (`knowledge_base.txt`) into the AI's system prompt. This acts as a robust policy manual, overriding the LLM's baseline training data and ensuring strict adherence to company-specific escalation and routing rules.

Multi-Turn Agentic Tool Calling: When the backend receives a ticket, it passes the context to the Z.AI GLM API. The AI may elect to pause its triage and call a defined backend tool (e.g., checking if a server is actually down). The backend executes the Python function, returns the real-time data to the AI, and the AI then produces a final, fact-based triage summary.

Structured Outputs: The GLM API is forced to return data matching a strict JSON schema (`response_format={"type": "json_object"}`). The backend parses this guaranteed structure:

- {
- "summary": "Payroll server is currently down. DB connection timeout.",
- "category": "software",
- "priority": "critical",
- "assigned_team": "Enterprise App Support"
- }

2.1.3 Dependency Diagram



2.1.4 Sequence Diagram (Multi-Turn Reasoning Loop)

1. **User:** Submits Incident via Frontend.
2. **Frontend:** Sends [POST /api/incidents](#) request to Backend.
3. **Backend:** Reads [knowledge_base.txt](#) and sends Prompt (Turn 1) to GLM API.
4. **GLM API:** Analyzes text and requests Tool Execution (e.g., [check_server_status](#)).
5. **Backend:** Executes requested Python tool and sends Tool Result back to GLM API (Turn 2).
6. **GLM API:** Returns final Structured Ticket Response (JSON).
7. **Backend:** Queries database for similar tickets sharing the same category.
8. **Backend:** Creates new ticket record, saves AI analysis and related tickets JSON array.
9. **Database:** Ticket stored successfully.
10. **Dashboard:** Retrieves updated ticket list and detailed AI analysis via [GET /api/tickets](#).

2.2 Technological Stack

2.2.1 Frontend

- **React.js (Vite):** Rapid UI development and dynamic state rendering.
- **Tailwind CSS & Lucide React:** Modern, responsive styling and iconography.
- **Axios & React Router:** REST API communication and client-side routing.

2.2.2 Backend

- **FastAPI (Python):** High-performance asynchronous REST API endpoints, agentic workflow loop management, and escalation handling.

2.2.3 Database

- **SQLite:** Lightweight persistence for ticket metadata, AI analysis outputs, workflow statuses, and serialized JSON arrays of related historical tickets ([related_tickets](#) column).

2.2.4 AI Service

- **Z.AI GLM API:** Core reasoning engine responsible for natural language understanding, function calling (tool use), and strict JSON generation.

3. Functional Requirements & Scope

3.1 Minimum Viable Product

#	Feature	Description
1	Incident Submission	Users submit incidents through support form or simulated email.
2	Agentic AI Analysis	AI utilizes internal tools and strict routing policies to summarize, categorize, prioritize, and assign tickets.
3	Ticket Creation & Linking	System creates structured ticket logs and dynamically links them to previous incidents of the same category.
4	Ticket Monitoring Dashboard	Dashboard displays an overview of ticket statuses and provides an in-depth modal exposing the AI's exact thought process.
5	Ticket Escalation	Unresolved tickets are escalated automatically after exceeding SLA time thresholds.

3.2 Functional Workflow

1. **Incident Submission:** User submits title, description, and department.
2. **AI Analysis (Agentic):** The incident is sent to the GLM API. The AI optionally calls backend tools, processes the routing rules, and returns a JSON schema containing the triage data.
3. **Database Lookup:** Backend identifies previously opened tickets with identical categories.
4. **Ticket Generation:** Ticket is saved with a unique ID, assigned team, priority, workflow state, AI summary, and an array of related tickets.
5. **Ticket Tracking:** Support staff view the centralized dashboard and can open a specific ticket to review the original description, AI summary, and linked incidents.
6. **Escalation:** SLA background triggers automatically flag stale tickets, updating their status to 'Escalated' and routing them to management.

3.3 Non-Functional Requirements

Quality	Requirement	Implementation
Performance	Ticket analysis (including tool execution) should complete within 5-8 seconds.	Async API requests to GLM and fast internal tool execution.
Reliability	Ticket creation must never fail due to bad AI string formatting.	Enforced native <code>response_format={"type": "json_object"}</code> .
Maintainability	Routing logic must be easy to update without touching code.	Policies abstracted out to a plain-text <code>knowledge_base.txt</code> file.
Scalability	Support multiple concurrent ticket submissions.	Stateless backend API design (FastAPI).

3.4 Out of Scope / Future Enhancements

- Real email inbox integration (IMAP/SMTP)
- Role-based authentication and Authorization (RBAC)
- Notifications through email or messaging apps (Slack/Teams)
- File attachment support for incident reports
- Advanced SLA policy management UI
- Vector-based Semantic Search for Similar Tickets (Currently matching via exact category strings)

4. Monitoring, Assumptions & Dependencies

4.1 Monitoring

The system monitors:

- Ticket creation success and related ticket linking
- Ticket status updates and SLA escalation triggers
- AI response validity and tool execution success

If AI analysis or tool execution fails:

- The exception is caught, and the ticket is safely created under a "Pending Manual Review" state to prevent workflow blockage.

4.2 Assumptions

1. Users provide somewhat coherent incident descriptions (though the AI is designed to parse unstructured/vague text).
2. Z.AI API endpoints are available within acceptable latency limits.
3. Departments and escalation routing rules mapped in the `knowledge_base.txt` are accurate and predefined.

4.3 External Dependencies

Tool	Purpose	Risk
Z.AI GLM API	AI reasoning, tool calling, JSON output	High – API failure impacts routing (mitigated via fallback state).
SQLite	Ticket persistence & relational lookup	Low – Highly stable for MVP scale.
FastAPI	Backend routing and orchestration	Low – Robust framework.
React/Vite	Frontend user interface	Low.

5. Project Management & Team Contributions

5.1 Project Management

For this one-week, We adopted a simple agile based task distribution methodology to ensure rapid progression and clear accountability for each member .Shared tools such as GitHub for version control and Google Docs for task tracking and documentation were used to maintain coordination. This lightweight methodology allowed the team to remain flexible, manage limited time effectively, and deliver a functional prototype within the hackathon timeline.

The allocation for task responsibilities are as follows:

Team Member	Assigned Task	Responsibility
Chiew Yi Jia	Project Planning	Define project scope, timeline, and coordinate daily progress updates
Tan Yi Shen	AI Workflow Design	Design the workflow logic, task routing, and automation process
Soo Desmond	Backend Development	Implement backend logic and integrate AI processing functions
Yan Calvin	Frontend / UI Development	Develop interface components and connect them with backend outputs
Yau Han Hong	Testing & Presentation	Test the prototype, prepare presentation slides, and document project outcomes

5.2 Recommendations

For future scalability beyond the MVP:

- Migrate SQLite to **PostgreSQL** to handle higher concurrency and native JSONB querying.
- Replace exact-category matching for similar tickets with **Vector Database Embeddings** to find semantically similar historical incidents.
- Add **Message Queues (RabbitMQ/Celery)** to process the agentic AI loops asynchronously, returning a ticket ID immediately to the user while the AI works in the background.
- Implement real email integrations and webhook notifications.

6. AI declaration

This document was authored solely by Team We-cooked-nen (Group 112) All core ideas, arguments, and conclusions were developed independently by the team. Generative AI was used strictly for structural refinement, formatting, and enhancing linguistic clarity. The final content remains the original intellectual product of the authors, who take full responsibility for its accuracy and integrity.